

Reviewer Pack — Minimal Geometric Entropy of Quantum States Bounds DPLL Sear...

Entry eae916e0d29f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 02:57:45 UTC

Minimal Geometric Entropy of Quantum States Bounds DPLL Search Tree Width

Entry ID: eae916e0d29f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 02:57:45 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Quantum States) **Field B** (complexity object): Boolean Function Complexity: DPLL Search Tree Width

Statement:

For every Boolean function f with n variables, the minimal geometric entropy of its quantum state representation $H_{\text{geo}}(\varphi)$ is polynomially bounded by the width of its DPLL search tree, specifically $H_{\text{geo}}(\varphi) \leq \Theta(\log(n)) * w_{\text{DPLL}}(f)$, where $w_{\text{DPLL}}(f)$ is the minimum size of a DPLL search tree for f .

Rationale (proposer's reasoning):

Quantum information theory has revealed non-classical properties that could potentially influence computational complexity. The geometric entropy of a quantum state provides a measure of its uncertainty and can be related to the structure of computational processes like DPLL search trees, which are known to have an exponential growth in size. This conjecture proposes a link between the geometric entropy of a function's quantum state and the width of its DPLL search tree, suggesting that quantum states with lower entropy might correspond to simpler proof structures.

Taxonomy category: cg_kw_andreev (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f2fc8cf990f27a80

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Spearman's rank correlation coefficient (ρ) between geometric entropy $H_{\text{geo}}(\varphi)$ and DPLL search tree width $w_{\text{DPLL}}(f)$ for 30 random Boolean functions is greater than or equal to 0.8, with a p-value less than 0.05.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal geometric entropy" AND "quantum state" AND "DPLL search tree width" - "Boolean function complexity" AND "geometric entropy" AND "DPLL search tree" - "quantum information theory" AND "DPLL search tree width" AND "geometric entropy bounds"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def dpll_search_tree_width(f):
        n = len(f)
        states = [{'assignment': [], 'unsatisfied_clauses': f}]
        while states:
            state = states.pop()
            if not state['unsatisfied_clauses']:
                return len(state['assignment'])
            unit_clause = next((c for c in state['unsatisfied_clauses'] if len(c) == 1), None)
            if not unit_clause:
                continue
            lit, polarity = unit_clause[0], unit_clause[1]
            new_state = {'assignment': state['assignment'].copy(), 'unsatisfied_clauses': state['unsatisfied_clauses']}
            new_state['assignment'].append((lit, polarity))
            for clause in new_state['unsatisfied_clauses']:
                if lit in clause:
                    clause.remove(lit)
                elif -lit in clause:
                    clause.remove(-lit)
            states.append(new_state)
        return float('inf')
```

```

def geometric_entropy( $\phi$ ):
    n = len( $\phi$ )
    Hgeo = 0
    for i in range(n):
        p_i = sum(1 for x in  $\phi$  if x[i] == 1) / n
        if p_i > 0 and p_i < 1:
            Hgeo -= p_i * math.log2(p_i)
    return Hgeo

n_values = [5, 10, 15, 20, 30, 40]
total_metric_value = 0
instances_tested = 0
n_max = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    for _ in range(5): # Each n tested 5 times
        f = generate_boolean_function(n)
        wDPLL = dpll_search_tree_width(f)
         $\phi$  = [f[i] for i in range(len(f))]
        Hgeo = geometric_entropy( $\phi$ )

        total_metric_value += Hgeo * wDPLL
        instances_tested += 1
        n_max = max(n_max, n)

        if wDPLL == float('inf'):
            conjecture_holds = False
            counterexample = "DPLL search tree width is infinite"
            break

    return {
        "metric_name": "Hgeo * wDPLL",
        "metric_value": total_metric_value / instances_tested,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r['metric_value'] for r in results) / len(results)
    std_value = math.sqrt(sum((r['metric_value'] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

```

```

if all(r['conjecture_holds'] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(r['seed'] for r in results if not r['conjecture_holds'])
    print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_681d95fa.py", line 92, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_681d95fa.py", line 64, in run_trial
    wDLL = dpll_search_tree_width(f)
            ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_681d95fa.py", line 31, in dpll_search_tree_width
    unit_clause = next((c for c in state['unsatisfied_clauses'] if len(c) == 1), None)
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_681d95fa.py", line 31, in <genexpr>
    unit_clause = next((c for c in state['unsatisfied_clauses'] if len(c) == 1), None)
                    ^^^^^
TypeError: object of type 'int' has no len()

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that it was unable to complete the computation necessary to evaluate the conjecture. | next: Re-run the test with appropriate error handling and debugging to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17540
2	preregistration	ollama_remote	glm4:latest	0	0	10172
3	novelty	ollama_remote	glm4:latest	0	0	16875

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	13295
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21867
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10668
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	38989
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	134189
9	judge	ollama_remote	glm4:latest	0	0	35229

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 298824 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/eae916e0d29f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/eae916e0d29f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/eae916e0d29f.tar.gz` (if generated)